

CricketMind: A Verification-Gated Multi-Agent Architecture for Multi-Stakeholder Cricket Analytics

Abstract—Automated cricket analytics is increasingly used in coaching, scouting and broadcasting, yet existing sports-analytics and agentic systems serve one audience with one task and report numbers they never verify against a trusted source. To address this gap, we introduce CricketMind, an agentic architecture that turns a batting-shot clip, or a text-only question, into a stakeholder-specific report. Two sanctioned engines, a statistics engine and a Monte Carlo what-if simulator, are the only components permitted to produce numbers; a supervisor router activates a subset of twelve stakeholder agents, and a faithfulness gate checks every figure an agent cites against the engines, regenerating failing agents within a bounded budget. We evaluate a LangGraph reference implementation whose components, apart from the router, are deterministic stand-ins. A closed-form cost model predicts the measured superstep count of seven scenarios exactly, with a worst case of 29 supersteps fixed when the graph is constructed. On twelve paraphrased queries that avoid every routing keyword, a keyword router activated the intended agent for none, while a locally hosted 7B language-model router activated it for nine, identically across three runs. The gate rejects fabricated citations but not correct figures attached to the wrong subject, showing that language-model agents require a structured citation contract.

Index Terms—multi-agent systems, large language models, sports analytics, cricket, verification, LangGraph

I. INTRODUCTION

Video footage from match broadcasts and team cameras can be segmented into short clips of individual batting strokes, and automatically reading these clips to judge a player’s technique and value is a practical and increasingly studied problem in coaching, scouting and broadcasting. The same stroke played by two different batters produces very different visual and statistical signatures, and turning raw footage into a single, trustworthy, numerically grounded analysis is where most existing systems fall short.

Most widely studied sports-analytics systems are built around a single narrow task for a single audience, such as forecasting how the players of a team will move [1], [2], spotting events in broadcast video [3], or training cooperative agents inside a simulated match [4], [5]. A newer line of agentic systems, built from several coordinating agents, has shown promise beyond motion prediction, for example in supply-chain decision-making [6], [7], and within cricket itself to help fans assemble a fantasy team [8]. However, each of these systems is built around one task and one answer format, provides no mechanism for checking its own numbers against a trusted source before reporting them, and offers no way to serve more than one kind of cricket stakeholder from the same underlying system. This matters because a report that sounds confident but was never checked against evidence can mislead

the very people—coaches, scouts, medical staff—who rely on it to make a decision. To address this critical gap, we introduce CricketMind, a single agentic system that turns a batting-shot video, or a plain cricket question with no video at all, into a checked, stakeholder-specific analytics report.

The main contributions of this research are outlined as follows:

- Development of CricketMind, an agentic architecture that accepts either a batting-shot clip or a text-only question. For a clip, a perception front end samples 15 frames, encodes them with a vision encoder and a temporal transformer, and classifies the shot, and a confidence gate with a bounded retry loop ensures that a low-confidence label is never passed downstream silently. A text-only question bypasses the perception path entirely.
- A shared, structured Shot DNA record and two sanctioned calculation engines, a deterministic statistics engine and a Monte Carlo what-if simulator, that hold exclusive authority to produce numbers, drawing evidence from two separate retrieval indexes: a precise ball-by-ball index and a broader historical index. Every other component must cite the engines’ values rather than compute its own.
- A dynamic supervisor router that activates only the relevant subset of twelve stakeholder-facing agents, covering audiences from coaches and players to medical staff, scouts and broadcasters. A router, either keyword-based or a language model, proposes the agents, and fixed rules decide: agents whose evidence does not exist for the query are removed, and at most four are activated.
- A faithfulness gate, the Faith Check, that compares every figure an activated agent cites against the values the two engines produced, sends only the failing agents back to regenerate, and withholds any finding that still fails once a bounded regeneration budget is spent, before the verified findings are composed into the final, stakeholder-specific report.
- As an evaluation of the reference implementation, an execution profile whose cost follows a closed-form model with a worst case fixed when the graph is constructed, and a comparison of a keyword router with a locally hosted language-model router on paraphrased queries.

II. RELATED WORK

This section reviews the multi-agent and reinforcement-learning systems that established cooperative agent modelling as a practical tool for sports and business analytics (Subsection II-A), before examining why none of these systems

supports a single, video-grounded, faithfulness-checked, multi-stakeholder cricket system such as the one proposed in this paper (Subsection II-B).

A. Multi-Agent Modelling and Sports Analytics Systems

Yeh *et al.* [1] established graph-based multi-agent trajectory modelling for team sports, combining a graph neural network with a variational recurrent network to obtain a permutation-equivariant model that outperformed fixed-ordering RNN and VRNN baselines on basketball and soccer tracking data. Teoh [2] extended this direction with CausalTraj, a temporally causal, likelihood-based model that, unlike prior work evaluated only on per-agent accuracy, explicitly targets the joint coherence of multi-agent forecasts, reporting the best recorded results on joint metrics on the NBA SportVU, Basketball-U and Football-U datasets while remaining competitive on per-agent metrics. Complementing these trajectory models, Ochinn *et al.* [3] released FOOTPASS, a benchmark for play-by-play action spotting over entire soccer matches that aligns broadcast video with multi-modal, multi-agent tactical data, motivated by using tactical knowledge as a prior to support computer-vision predictions. At the environment level, Kurach *et al.* [4] built the Google Research Football Environment, an open-source, physics-based simulator supporting single- and multi-agent reinforcement learning, and reported baseline results for IMPALA, PPO and Ape-X DQN across benchmark scenarios of varying difficulty. Brandão *et al.* [5] applied multi-agent reinforcement learning with self-play to learn both low-level control and strategic decision-making for teams of wheeled robots in the IEEE Very Small Size Soccer league. None of these systems, however, provides a mechanism for verifying a numeric output against a trusted evidence source, and none is designed to serve more than one downstream audience from the same architecture.

B. Agentic Systems and the Verification Gap

A more recent line of work applies coordinating agents, increasingly built on large language models, to business and fan-facing decision problems. Gupta [6] proposed an agentic AI framework for supply-chain management in which multiple coordinating agents support decentralised decision-making across supply-chain operations. Hazenberg *et al.* [7] benchmarked three multi-agent reinforcement learning algorithms, MADDPG, MADQN and QMIX, against static rule-based pricing in a simulated supply chain informed by real e-commerce data, finding that rule-based agents achieved the highest fairness and price stability but lacked competitive dynamics, while MADDPG balanced market competition with relatively high fairness. Closest to the present work, Bhatnagar [8] introduced FanCric, a multi-agent system of coordinating LLM agents that combines structured and unstructured data to select fantasy cricket teams, and evaluated it against the collective selections of roughly 12.7 million contest entries and a simpler prompt-engineering approach, reporting promising exploratory results. While these agentic frameworks show that coordinating several specialised agents is a promising way

to structure real-world decisions, none of them begins from raw video evidence, none verifies its agents' numeric claims against a shared, sanctioned source of truth before a report reaches the user, and none is designed to route a single underlying analysis to more than one type of stakeholder. To our knowledge, no existing study combines video-grounded shot classification, retrieval-backed statistical evidence, dynamically routed multi-stakeholder agents and a dedicated faithfulness-checking gate within a single cricket analytics system, a gap this paper addresses through CricketMind.

III. METHODOLOGY

CricketMind is built as one connected graph that answers two kinds of cricket question: a question about a specific video clip of a shot, and a question that is only text, with no video at all. Fig. 1 shows the architecture; the node identifiers N1–N13 used below refer to it.

A. Two Entry Paths and Perception

When a query is submitted, the system first checks whether a video clip is attached. If it is, the clip is uniformly sampled into fifteen frames (N1), enough to capture the whole shot from the backlift to the follow-through without spending effort on near-identical frames. A vision encoder (N2, EfficientNetV2-S) turns each frame into a 1280-dimensional feature vector, and a small temporal transformer (N4, two layers, four heads) fuses the fifteen vectors. Because self-attention lets the first frame of the shot directly influence the last, the fused representation is designed to separate shots that look similar frame by frame but differ in the order and speed of motion, such as a sweep and a reverse sweep. A classifier head (N5) then predicts one of the fifteen shot classes of CricShot10k [9] together with a confidence score.

A confidence gate adjudicates that prediction. A label with confidence of at least 0.70 is accepted. Below that, the system does not accept the result but re-samples the clip's frames and classifies again, at most twice; if the confidence is still too low, the query terminates with an explicit low-confidence status, before any agent is invoked, so an unreliable label is never passed forward silently. At the same time, and on both paths, a text encoder (N3) embeds the query and extracts the filters it implies—the player, the bowler type, the match phase, and any shots the question names—so the system knows which player, shot or situation is being asked about even when there is no video.

B. The Shot DNA

The accepted shot label, the query filters and posture measurements such as weight transfer, bat-face angle and stride length are combined into one structured record, the Shot DNA (N6). A text-only query skips the entire video path and arrives at this same record directly, carrying only the information taken from its own text. In that case the posture measurements are left out rather than filled in with a guess, because inventing a measurement that was never taken would quietly poison the very record that the rest of the system is built to trust. The Shot

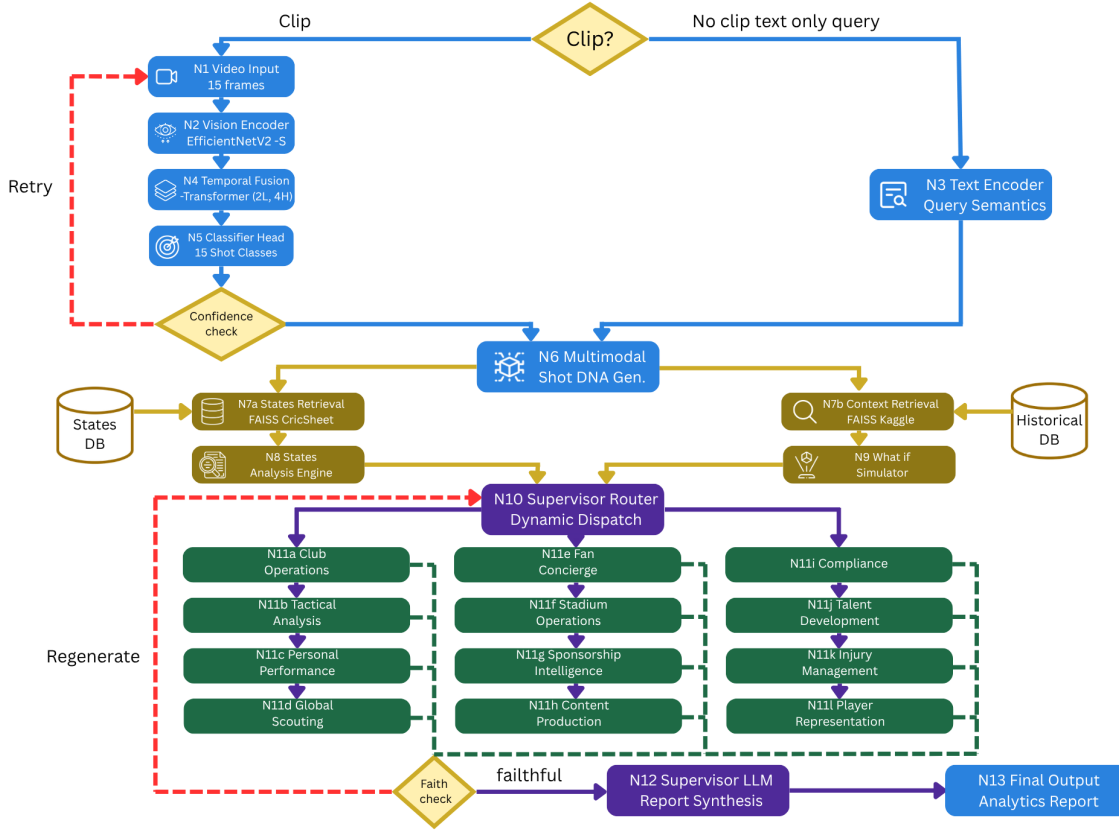


Fig. 1. The CricketMind architecture. A query carrying a clip enters the perception path (N1–N5); a text-only query bypasses it and reaches the Shot DNA (N6) directly. The text encoder (N3) runs on both paths. Red dashed edges are the two bounded correction loops: the confidence retry and the Faith Check regeneration.

DNA is the only interface between perception and analysis: every later node reads it, and none touches the video again.

C. Retrieval and the Sanctioned Engines

The Shot DNA is used to search two separate indexes: a precise ball-by-ball index (N7a) that returns the specific deliveries matching the query, and a broader historical index (N7b) that returns career-level context, so the analysis is neither too narrow nor too general. Two shared calculation engines then take over. The statistics engine (N8) computes exact aggregates, such as strike rate and dismissal rate per shot, from the retrieved deliveries. The what-if simulator (N9) answers questions about hypothetical situations by Monte Carlo simulation, reporting how likely an outcome is together with a 95% confidence interval. These two engines are the only parts of the system allowed to produce a number; their outputs form the *sanctioned facts* of the query, and every other component must cite them rather than compute its own.

D. Routing

A supervisor router (N10) decides which of twelve specialised agents answer the query. Each agent serves one cricket stakeholder: franchises, coaches, players, scouts, fans, stadiums, sponsors, broadcasters, governing bodies, academies, medical teams and player agents. Routing has two stages. First,

a router *proposes* agents. The keyword router scores each agent by the words of its keyword list that appear in the query, plus two structural signals from the analytics: a low visual confidence, which suggests a technique question, and a large strike-rate spread between shots, which suggests a tactical one. The language-model router instead asks a language model, which sees each agent’s stakeholder name and keyword list and must reply, under a fixed JSON schema, with one to four agent identifiers from the registry. Second, fixed rules *decide*, identically for either router: an agent whose cited facts do not exist for the query is removed, at most four agents are kept, and if none remain a default pair (Coaches and Players) answers. On a text-only query, for instance, the agents focused on personal technique and player development are removed, because their answers would depend on posture measurements that were never taken; the report is shorter but still fully backed by evidence. Because the rules follow the proposal, the language model can suggest which agents answer but cannot activate an agent whose evidence does not exist, nor exceed the cap.

E. Citation Contract and the Faith Check

Every agent returns its answer together with a citation record that lists each figure it uses and the sanctioned fact it was

taken from. Before any answer is shown to the user, the Faith Check compares every cited figure with the value the engines actually produced. If an agent cites a figure that does not match, only that agent is sent back to regenerate its answer from the real evidence, at most twice; if it still fails, its finding is withheld and the report states that it was withheld. The gate therefore guarantees that no cited figure contradicting the engines reaches the user. It checks citations, not prose: a correct figure attributed to the wrong subject, or a figure an agent states without citing it, lies outside what it can detect, which is why the citation record must be enforced on the agents’ output format.

F. Synthesis

Finally, the verified findings are combined into one readable report that directly answers the user’s question (N12), and the report is returned with a status (N13): `ok`, `low_confidence` if the clip could not be classified, or `unverified_claims_dropped` if a finding was withheld.

G. Implementation Status

We implement the architecture as a LangGraph state graph with one node per component of Fig. 1, 30 nodes in total. In the implementation evaluated in this paper, node bodies are deterministic stand-ins. No video is decoded: the classifier returns a configurable label and confidence, which lets the confidence gate and the retry loop be exercised on demand. Retrieval operates over a small synthetic corpus of 150 deliveries and four historical records with known ground truth. The stakeholder agents generate their answers from templates over the sanctioned facts, emitting the same citation record that a language-model agent would be constrained to produce, and the report is composed from a template. The one exception is the router: alongside the keyword router we evaluate a language-model router, Qwen2.5-7B served locally through Ollama (Section IV-B). Evaluating the accuracy of the perception front end is outside the scope of this paper. Values in the examples illustrate the flow of data and do not describe real match data.

IV. RESULTS

A. Execution Profile

Node bodies are deterministic, so repeating a query reproduces its execution exactly. We therefore characterise the cost of the architecture across situations rather than across repetitions, using the evaluation query “*Compare Kohli’s cover drive vs his pull shot against fast bowlers in the powerplay*”. LangGraph executes a graph in *supersteps*—rounds in which every scheduled node runs concurrently—and we record the nodes executed in each superstep from the framework’s debug event stream. Table I reports, for seven scenarios, the number of supersteps, the total number of node executions, and the number of stakeholder-agent invocations; in deployment, each invocation is a language-model call. Fabrication is injected by configuring a named agent to corrupt a cited value.

TABLE I
EXECUTION PROFILE OF THE EVALUATION QUERY. r : VISION RETRIES; g : REGENERATIONS. IN THE WORST CASE THE CLIP IS REJECTED TWICE AND EVERY ACTIVATED AGENT FABRICATES ON EVERY ATTEMPT, SO ITS FINDINGS ARE WITHHELD. EACH SCENARIO WAS RUN ONCE: NODE BODIES ARE DETERMINISTIC, SO REPEATED RUNS ARE IDENTICAL.

Scenario	r	g	Steps	Node runs	Agent calls
Video, confident	0	0	12	16	2
Text-only	0	0	9	11	1
Video, 1 retry	1	0	17	21	2
Video, 1 fabrication	0	1	15	19	3
Text-only, 1 fabrication	0	1	12	14	2
Retries exhausted	2	0	15	16	0
Worst case	2	2	29	35	6

a) *Cost model*: Each event has a fixed superstep cost that can be read from the topology in Fig. 1. A query that reaches the report takes $S_0 = 12$ supersteps with a clip and $S_0 = 9$ without one. Each vision retry adds five (the retry node and the four-stage vision chain), each regeneration adds three (preparation, the failed agents in parallel, and a second Faith Check), and withholding unverified findings adds one. Hence

$$S = S_0 + 5r + 3g + d, \quad S_{\text{degrade}} = 4 + 5r + 1, \quad (1)$$

where $d \in \{0, 1\}$ records whether findings were withheld, and S_{degrade} applies when the retry budget is exhausted before classification succeeds. Equation (1) predicts the measured superstep count of all seven scenarios exactly, confirming that on these paths the implementation executes the rounds the topology prescribes.

b) *Bounds*: Substituting the loop budgets $R_{\max} = G_{\max} = 2$ into (1) gives a worst case of $12 + 10 + 6 + 1 = 29$ supersteps, fixed when the graph is constructed; the worst-case scenario reaches this bound exactly. Because agents dispatched together execute in a single superstep, the bound does not depend on how many agents the router activates. Agent invocations are bounded separately, by $A_{\max}(1 + G_{\max}) = 4 \times 3 = 12$ for an activation cap of $A_{\max} = 4$. This bound holds by construction and was not reached in our runs: the router activated two agents for the evaluation query, so at most six invocations were possible.

c) *Where the cost falls*: Two properties follow from Table I. First, failures in perception do not propagate cost to the agent layer: a vision retry adds no agent invocations, and a clip that exhausts its retry budget terminates after 15 supersteps with none. Second, verification is targeted. A single fabrication re-invokes only the offending agent, adding three supersteps and one invocation on both the video and the text-only path.

Supersteps measure the length of the sequential dependency chain, not wall-clock time. In deployment, latency will be dominated by the encoders and by agent language-model calls, which these counts bound but do not time.

B. Query-Conditioned Routing

We compare the keyword router and the language-model router of Section III on the same queries. The language

TABLE II
KEYWORD ROUTER VERSUS LLM ROUTER ON THE 29 PROMPTS. A TARGET
COUNTS AS ACTIVATED ONLY WHEN THE ROUTER CHOSE IT, NOT WHEN IT WAS
REACHED THROUGH THE DEFAULT PAIR.

	Keyword	LLM
Keyword prompts: target activated	12/12	12/12
Plain prompts: target activated	0/12	9/12
Fell back to the default agents	14/29	2/29
Agents activated per prompt (mean)	1.79	1.41
Identical routing in all 3 runs	–	29/29
Routing time per prompt (mean)	–	3.0 s

model is Qwen2.5-7B, served locally through Ollama with temperature 0 and a fixed seed; for a fair comparison it receives exactly what the keyword router knows, each agent’s stakeholder name and keyword list. The instructions were fixed before the comparison was run, and the same rules follow both proposals, so the bounds of Section IV-A hold for either router.

a) *Prompt set*: For each of the twelve stakeholders we wrote two questions asking for the same thing: a *keyword* version that uses the agent’s trigger words (“Is Kohli’s workload a fatigue risk before the final?”) and a *plain* version that avoids all of them (“Is Kohli too tired or sore to play the final safely?”). Five further prompts cover a question with no signal, one naming six stakeholders at once, the running example, and two text-only questions—29 in all. The notebook verifies that every keyword prompt contains its target’s trigger words and that no plain prompt contains any. Each prompt was routed once by the keyword router, which is deterministic, and three times by the LLM router.

b) *Results*: Table II summarises the comparison. Both routers activate the intended agent for all twelve keyword prompts. On the plain prompts the keyword router activates none of the twelve: finding no trigger word, it falls back to the default pair on every one, and on 14 of the 29 prompts overall. The LLM router activates the intended agent for nine of the twelve plain prompts (Table III) and falls back only twice. Both fall-backs are text-only prompts for which the model proposed only agents that cite a posture measurement; with no clip there is no posture, so the evidence rule removed them. The LLM router is also more selective, activating 1.41 agents per prompt against 1.79. On the prompt naming six stakeholders, the keyword router’s four-agent cap, with ties broken by registry order, discarded Sponsors, Medical Teams and Broadcasters; the LLM router kept all three.

The LLM router’s answers were identical in all three runs for all 29 prompts, and a routing call took 3.0 s on average (2.8–3.6 s) on a single consumer GPU. Across all 87 LLM-routed runs, no activated agent lacked its evidence and no run exceeded the cap; this holds by construction, since the rules follow the proposal, and the runs confirm it.

c) *Where the LLM router fails*: The three plain prompts it missed are informative. Asked how to dismiss Kohli cheaply against spin, it chose the batter’s perspective (Players) instead of the coaching staff’s; asked which home game a supporter should attend, and how to teach teenage batters a stroke, it again chose Players rather than Fans or Academies. The router

also sees only the query text. For the running example the keyword router additionally activated Coaches because of a 30-point strike-rate spread in the analytics, a signal the language model never receives.

d) *One query through the graph*: Table IV traces two queries through every stage, recorded from the framework’s update stream rather than written by hand. The same pipeline extracts different filters, retrieves different evidence (100 against 80 deliveries), computes different statistics, and activates different agents, and every figure an agent cites is checked against the engines before release. The trace also exposes a cost: for the medical query, the strike rates and the dismissal simulation are computed although the activated agent cites only the sample size, because every analytics stage runs before routing.

C. Boundary of the Faith Check

To locate what the Faith Check cannot catch, we ran the evaluation query with the Coaches agent’s answer altered in four ways. The honest answer, “the Pull returns SR 112.0 against the Cover Drive’s SR 142.0”, passes. A fabricated citation (*best_sr* = 999.0) is rejected, regenerated once, and never reaches the report. Two further answers, however, pass and reach the user. In the first, the sentence swaps the two real strike rates onto the wrong shots (“the Pull returns SR 142.0 against the Cover Drive’s SR 112.0”) while citing exactly the same values as the honest answer. In the second, the agent adds a figure it never cites (“61% of his runs from boundaries”). The gate verifies citations, not prose. Its guarantee therefore holds only if the citation contract is enforced on the agents’ output: every numeric token must be emitted as a citation that carries its subject, through a JSON schema or a tool definition rather than free text, so that the sentence is rendered from verified citations. Without that constraint the guarantee degrades silently, since an uncited figure is indistinguishable from a verified one.

V. LIMITATIONS AND FUTURE WORK

Every component other than the router is a deterministic stand-in evaluated on a synthetic corpus, so the results characterise the architecture’s control flow and guarantees rather than the quality of its analyses; the accuracy of the perception front end is outside the scope of this paper. The routing comparison uses 29 author-written prompts whose targets describe the prompts rather than serve as ground truth, and twelve plain prompts are too few for a benchmark, so it is indicative; its stability was measured on one machine. The Faith Check’s guarantee is conditional on the citation contract (Section IV-C), and a query about a player with no deliveries in the corpus is not yet handled gracefully.

Future work replaces the stand-ins: language-model agents constrained to emit subject-keyed citations, retrieval over real shot-annotated ball-by-ball data, and counterfactual questions such as a changed batting order. Routing before the analytics would compute only the statistics the activated agents cite, and giving the language-model router a summary of the evidence

TABLE III

THE TWELVE PLAIN PROMPTS, WHICH AVOID EVERY TRIGGER WORD OF THEIR TARGET AGENT, AND THE AGENTS THE LLM ROUTER ACTIVATED. **BOLD:** THE TARGET WAS ACTIVATED BY THE ROUTER’S OWN CHOICE. THE KEYWORD ROUTER ACTIVATED THE DEFAULT PAIR (COACHES, PLAYERS) FOR ALL TWELVE.

Written for	Plain question (no trigger words)	LLM router activated
Franchises	Should our team keep Kohli next season, or let him go to save money?	Franchises , Players, Player Agents
Coaches	How do we get Kohli out early when he faces spin?	Players
Players	Has Kohli’s batting got better or worse this season, and what should he change?	Players
Scouts	Find young overseas batters who could become the next Kohli.	Scouts , Players
Fans	I want to watch Kohli bat live; which home game should I go to?	Players, Franchises
Stadiums	How should the ground staff get the wicket ready if we want Kohli to struggle?	Stadiums , Medical Teams
Sponsors	Is paying to put our company name on Kohli’s bat good value for money?	Sponsors
Broadcasters	What on-screen visuals should the TV team show when Kohli walks out to bat?	Broadcasters
Governing Bodies	Did Kohli break any of the game’s official regulations in this innings?	Governing Bodies
Academies	How can we teach teenage batters to hit the ball through the covers like Kohli?	Players
Medical Teams	Is Kohli too tired or sore to play the final safely?	Medical Teams , Players
Player Agents	How much money should Kohli ask for in his next deal?	Player Agents

TABLE IV

TWO QUERIES TRACED THROUGH THE GRAPH. A: “COMPARE KOHLI’S COVER DRIVE VS HIS PULL SHOT AGAINST FAST BOWLERS IN THE POWERPLAY”. B: “IS KOHLI’S WORKLOAD A FATIGUE RISK BEFORE THE FINAL?”. EACH CELL IS WHAT THE STAGE WROTE TO THE SHARED STATE; AGENT CELLS LIST EVERY FIGURE THE AGENT CITES. STAGES WHOSE OUTPUT IS IDENTICAL FOR BOTH QUERIES (PERCEPTION, SHOT DNA, CONTEXT RETRIEVAL, OUTPUT) ARE OMITTED.

Stage	Question A	Question B
N3 reads the question	bowler_type=pace, phase=powerplay, player=Kohli, shots=Cover Drive+Pull	player=Kohli
N7a finds deliveries	100 matching deliveries	80 matching deliveries
N8 computes statistics	Cover Drive SR 142.0 (50 balls), Pull SR 112.0 (50 balls)	Cover Drive SR 130.0 (80 balls)
N9 simulates	6 consecutive deliveries targeting the pull: 39.4% out (95% CI 38.7%-40.1%)	6 consecutive deliveries targeting the cover drive: 13.9% out (95% CI 13.4%-14.4%)
N10 picks the experts	wakes Players, Coaches	wakes Medical Teams
N11 agents answer	Coaches: best_sr=142.0, worst_sr=112.0, p_pct=39.4; Players: best_sr=142.0, worst_sr=112.0, weight_transfer=0.88	Medical Teams: sample_size=80
Faith Check	all cited figures match the engines	all cited figures match the engines

would let it use the signals the keyword router already reads. A further direction is to measure the cost of containment: how report completeness and agent invocations change as agents fabricate more often, a trade governed by the regeneration budget.

VI. CONCLUSION

This paper introduced CricketMind, an agentic architecture that answers cricket questions, with or without a batting-shot clip, through twelve stakeholder agents whose every cited figure must come from one of two sanctioned engines and is verified before release. Implemented as a LangGraph state graph, its cost followed a closed-form model that predicted all seven measured scenarios exactly, with a worst case of 29 supersteps and 12 agent invocations fixed when the graph is constructed. On twelve paraphrased queries that avoid every routing keyword, a locally hosted 7B language-model router activated the intended agent nine times where a keyword router activated it none, with identical decisions across three runs, and the fixed rules that follow its proposal preserve every bound. The Faith Check rejected fabricated citations but not a correct figure attached to the wrong subject or a figure left uncited, making a subject-keyed citation contract a requirement for language-model agents rather than an implementation detail. CricketMind thus offers a verifiable foundation on

which the stand-in components can be replaced by learned ones without weakening its guarantees.

REFERENCES

- [1] R. A. Yeh, A. G. Schwing, J. Huang, and K. Murphy, “Diverse generation for multi-agent sports games,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2019.
- [2] W. Z. Teoh, “Coherent multi-agent trajectory forecasting in team sports with CausalTraj,” in Proc. AAAI Workshop on AI for Time Series (AI4TS), 2026, arXiv:2511.18248.
- [3] J. Ochin, R. Chekroun, B. Stanculescu, and S. Manitsaris, “FOOTPASS: A multi-modal multi-agent tactical context dataset for play-by-play action spotting in soccer broadcast videos,” Comput. Vis. Image Underst., 2026, arXiv:2511.16183.
- [4] K. Kurach et al., “Google research football: A novel reinforcement learning environment,” in Proc. AAAI Conf. Artif. Intell., 2020.
- [5] B. Brandão, T. W. De Lima, A. Soares, L. Melo, and M. R. Maximo, “Multiagent reinforcement learning for strategic decision making and control in robotic soccer through self-play,” IEEE Access, vol. 10, pp. 72628–72642, 2022.
- [6] D. Gupta, “Autonomous agentic AI frameworks for supply chain resilience and decision-making,” in Proc. Int. Conf. Innov. Practices Technol. Manage. (ICIPTM), 2026.
- [7] T. Hazenberg, Y. Ma, S. S. Mohammadi Ziabari, and M. van Rijswijk, “Multi-agent reinforcement learning for dynamic pricing in supply chains: Benchmarking strategic agent behaviours under realistically simulated market conditions,” arXiv preprint arXiv:2507.02698, 2025.
- [8] M. Bhatnagar, “FanCric: Multi-agentic framework for crafting fantasy 11 cricket teams,” arXiv preprint arXiv:2410.01307, 2024.
- [9] M. K. Dihan, Abdullah, Amina, and S. Ahmed, “CricShot10k: A large-scale video dataset for cricket shot classification,” IEEE Access, 2026, doi:10.1109/ACCESS.2026.3663220.